

Baca ini untuk guide membuat folder :

1. interpretable context methodology. That's the name of my paper that I published about 3 months ago. And now thousands of people and hundreds of GitHub stars as well as over 30,000 people in the community are all using my methodology to build their own systems with AI, but they're not building multi-agentic frameworks and all these absurdities. They're building folders and markdown files on their computer and getting huge results from it. Now, I have a quite a few videos on that, but today I wanted to talk less about the methodology and more about what I've been building, what me and my team have been building as a future thing. What can you actually build using this methodology and what happens when you bring in some traditional software programming into it? Now, for those of you who are new to my channel, essentially my methodology is about structuring folders, structuring markdown files. that's very similar to the LLM wiki that Kaparthy, the founder of OpenAI and uh currently just moved to Anthropic to teach there uses and is very big in Anthropic's actual use cases. They are very big around skills which is the idea that the right amount of scripts, processes and ideas can all be kind of broken down into just plain text and folders and the agent can navigate that to get the context rather than using really complicated workflows or infrastructures and harnesses. You can create one really good harness for a single type of model and then that model can then navigate structure as it needs to. Instead of creating rag that pulls everything in, you just give it access to normal databases. You give it access to folder and context structure. If you haven't read anything or dive into this process, I do a deep one with clawed code in my video right here that you should see tagged. Essentially, it just looks into how to use this, how I use it. And a lot of people are seeing 20 30 40% token reduction and really just getting outcomes a lot faster without a lot of infrastructure. Even further, you're able to use it to really build out teams and build out processes that traditionally would have taken, you know, a vendor 6 months and an entire infrastructure update versus just working with them. And as the people learn how to use the AI, they're building the structure for you. Your team become the vendor. And then you can hire technical talent or bring in your own technical talent to amplify it. Look at the security and governance behind it. But all of that aside, let me chat about transcripts, about dialogue, about content, and why it's so important to this entire process. Now, this is an actual slide that I gave to a real enterprise company that's bringing in, I think, 60 million a year or something like that, and they have learned and used this process. One of the biggest things that I try to share with people when we're trying to understand how to use AI, how to use these tools in your workflow is my layer process. Right? Layer one is where you're just chatting with a chatbot. You're copy and pasting. The layer one is this concept where I started years ago when I first started messing around with language processing systems and a lot of people are today. It's where you log into Chat GPT or Claude and all you're doing really is just kind of pasting in ideas, right? You're just chatting back and forth in it. You're copying and pasting. You're throwing in concepts in there. Um, you're just asking it basically questions and things like that. What you actually want to really understand is the amount of effort to get something back is low and you can get some pretty decent things, but the amount of output and impact that you get is actually very weak, right? It's not going to be huge. You can get some very good stuff, but earlier 3 years ago, it was very challenging to get any sort of outputs. And in order to really get a better output out of it, you have to have longer and longer conversations. as in you're

essentially adding multiple chats. You have saved prompts. Maybe you have a tone style. And we see a lot of people sharing prompt libraries. They have shared prompts. Maybe they're adding the prompts in little by little. A lot of other like kind of integration tools do this where they're grabbing a prompt and giving it to the AI automatically. It's doing chain of loop processing. And these skills are actually the layer 2. someone else went through that layer 2 workflow, found the right amount of prompts that you actually want to be able to give in what order you want to give them and kind of handled that as a skill. But then skills evolve even further and you realize you only need certain parts of a skill at a certain time. You don't want to overload the context window and you want to start creating automations. That's where you start leaning into level three or layer three, which is where you're actually doing this huge workflow. You're taking multiple skills, multiple prompts, multiple workflow processes, maybe even multiple AI, and actually linking them together. Then they realized that you want to do even more steps. You want these agents. And that's where you have these other markdown files or scripts in many areas, right? people actually put Python for people to actually run, for the AI to actually run itself to be able to handle the creation of stuff because they needed more determinism. They wanted the output to be the same every time. And so if they could package all of that into a prompt, all of this could be uploaded and sent in the right order, that's where skills come from. And this allows them to then, in this case, create more skills. But you have hundreds of skills available and if you have claude installed on your computer you can actually come through and look through these skills deeply. Some of these skills right for algorithmic art a skill for creating art with like kind of an algorithmic process has an HTML viewer and a template built into it. Maybe you're trying to create a doc file in markdown. You have the actual skill explanation. This is how you do it. These are the prompts they put in order. Traditionally, two years ago, I was putting these prompts in manually. First, I would I knew that, okay, if I told the chatbot this in a certain order, right? I come in and I paste this here, right? It would do that process. I would wait for its output, verify that it's right, and then I would move on to the next step, which is, okay, well, then we need to edit that and have these things. And over time, we realized we could drop that into the AI all at once and it would recognize that. But then we could drop all of it into the AI at once and tell it to reference certain areas or actually use commands and the single agent could then do those things. Right now, traditionally before the way they solved that is using lang chain or semantic kernel or all of these things to do these actions separately or to inject these at the right time in the thinking process of the AI. But now we can just break down the thinking process into folders and structures and trigger them as they're needed, which is what these folder skills and processes are doing. But again, a common theme. There's a reason I'm describing all of this. All of these skills, all of these folders and markdown files all have one core theme, discussion and dialogue. They all came from an initial layer one of discussion and dialogue breakdown. At layer 1, you're pasting a prompt into chat GPT and then copying the output or going through iterations of an output. At level two, you already expect what the output is from level one. So you have refined prompts and processes that are automatically being injected in. At level three, you're doing all of the level two automated multiple times so that you can then come in and look at these processes. And that goes for code review, data reporting, writing and drafting, all of those. That's what we really want to understand here. But it all comes from dialogue. conversation has the structure we're looking for. The intent is carried

in the conversation. And instead of just describing this randomly, here's a tool K, who is one of my uh co-founders and a really brilliant person certainly studying at the University of Edinburgh, created to really help exemplify this process. So, imagine here on the left, this is the chat, and we're going to break this down to the simplest process. This is a chat that may have happened forever ago. And the idea is, can you tighten this paragraph? It feels too worry. Climate change blah. And you put in the paragraph in there. Now, in the past, right, the agent would just say, here's a new one. And then we would say, hey, this feels weirdly formal. Now, the original had more of a conversational rhythm. And so, the agent does something, and then it's this constant back and forth until we finally get the output we wanted out of this. Now everywhere along this conversation even in this simple one there is a chain of decisions being made and if we actually extract those chain of decisions using K's tool here on the left is the decisions made by the user here on the right are the decisions made by the model. Now when we look in the goal was to tighten a paragraph. All of this is all of the goals and focus and as you can see it links directly here. Can you tighten this paragraph? It feels too wordy. Now, we set up constraints along the process and at different points. Perhaps there was a constraint saying, "Hey, there was too much wordiness. We want to perceive the rhythm or conserve what the rhythm in the original paragraph was." Maybe we want to have it more styled or voked. Right here, we want it scarier, way behind. The voice we're going for is more measured, right? in that same sentence. Maybe we want to match the entire voice reference. The assumption is that we need it to match the original. And both the model and the AI follow this assumption. And don't worry, we're going to go over to this right one in a second, but I really want you to understand this. Finally, the goal of what we're making, the process is an actual blog post or a reference, and we give an example of what that should be. There's also dialogue happening, right? Questions that are happening. I think there's a way in which to do these things. When we go and look at the model over here, the same thing. The model sees the goal. It sees the constraints, but it is making decisions on those constraints. It's coming through and saying, "Hey, this retains a cleaner structure. I made this decision for this goal. I made this decision for this goal." And it's also making assumptions along the whole way. We create all of this data. We look at this and instead what we can do is we can create a system of tone, of style, of ideas that capture all of these problems and constraints already. So that when I say tighten this paragraph, this tree of processes, this tree of prompts is called the first time. Again, thinking about where we're at. Originally, it was layer two. We had to sit there and describe these processes, verified outputs. we captured those turn those into maybe markdown files or structure and then the original request becomes a layer one process where I'm able to simply say can you tighten this paragraph again this also is the same thing for everything let's say we're doing the opposite side we're actually refactoring code and I actually say hey my parthonon my my Python is treating code a certain way I noticed that this parser can you kind of take a look and fix it possibly and of Of course, the assistant assistant comes in, says what it thinks its fix is. Maybe we say some sort of reference because we're an expert in the field. Maybe this is how they did training early on is this kind of back and forth. And you eventually get to the conclusion, the goal. When we actually extract that data, right, you end up getting the same thing. You have goals, then you maybe even have sub goals, right? Can we just patch the empty cell instead and keep the original function? You have constraints

within this dialogue and you have processes that live inside of it as well. Same for the model. The model now has the goal. It's coming through and it's making decisions on that goal. And all of it can be tracked through dialogue. That goes for anything. Database schemas, you can track the decision making and goal processes where and when it was saved within dialogue. All of this becomes either training data or even better actual markdown files that can be used later converted into a methodology here. Right? And it's important to understand this not necessarily that you could use this tool. We might make this tool open source right now. It's being used for a research project at the University of Edinburgh. Um but the idea here isn't for you to you all use this tool right now. The idea is what this tool shows about dialogue, about prompting, and about engineering context, where and what you want to engineer context, at what levels you do. And that's what has informed a lot of the work I'm doing. And an example is one here. My dialogue with the AI made me realize it sucks at certain processes of reading and research. So, I created a kind of pillar of idea and my voice and my patterns and constraints that capture my process of writing and researching as well as my tone. No matter what project I'm working on, I use these processes. So, that's all just captured in a very simple short prompt, right? that is referenced whenever I need it through the initial prompt which is my Claude MD that gives folder structure of wherever I'm hiding and wherever that voice is I say hey every session I'm going to have certain things built in my voice and Claude MD directly says how I speak and teach and just by me simply saying right without creating any crazy agentic architecture what is my voice and tone Now, if I had asked an AI model that was at layer 1 or 2 years ago, it would have no idea. I'd have to paste a whole bunch of prompts in there, setting all in. If I was one at layer two, it would have to read the whole database, go through everything, finally find the structure. At layer three, I've solved the problem of it navigating. So, it can easily come in, find my voice and tone markdown file, actually look through and see how I teach in tone with one simple prompt. This prompt is actually thousands of prompts, which if any of you have watched my Python video, 12,000 lines of Python, it's the same thing. When I type hello world, it is thousands and thousands of lines of code that are executing to print hello world in Python. If you haven't seen that, I highly recommend it. It's very useful for this kind of conceptual. This same thing is happening now with these processes. This is the next layer of software engineering. But with every layer comes requirements, comes processes. And that is exactly what I'm encoding here. These processes keep expanding until I finally have an output. In this case, my output was the ability to not only write and research in my own area. So I had a company NLP Logix that wanted me to create videos for them. I spent time building ideas and my tone around it. and I said, I want to make three videos based on three of their products. So, it goes through, it has access to my research and creates a research folder on each of the products, the angle, where they are, what type of hooks might work, what type of outlines and materials. And again, it's all markdown files. These are just more prompts to be added to context in the future, whether it's this AI or any other AI that I'm using to read these folders. And then all I have to say is cool. Now do the research based off of all of that. Then with a third prompt, again, I'm only technically said two prompts so far in this whole process of getting all of this research, all of this data, all of this possibility. And it was able to create me these scans, these source materials to then move on to the next step in which I say cool, let's make a script for each. Now before if I had said let's make a

script it would try to assume what a script should be. I have pillars and processes on how script should be written where and why. Think of these as my own personal skills. So it goes through it reads my voice right. It automatically knows my voice constraints. It does the actual processing. It knows where the documents need to go. Again with no crazy injection harnesses. Yes, the Claude code agent has harnesses in its own right, but that's the point. I don't need to reinvent the wheel. It's already been invented. Then boom, I get scripts. And guess what? I say, can you generate the audio for those scripts? I already had a workflow that pulls from my 11 labs on my voice that's already been trained, pulls all that data, and says, "Cool, we're going to go ahead and produce that audio." Then I have the workflow which at this point I can open up a fresh conversation if I want to save tokens. All of the process has been saved. All of the context is in the structure. I don't need to inject it back in at the right time. It can pick right up off and I can then say, "Hey, let's make a video animation." And rather than show you the code, let me just share with you the animation. It took my voice and it took my placeholder files that I had already created for animations for this entire project and simply built from them rather than generating new ones. And it took my voice, chopped it up, and allowed me to create what you see here, insurance company that emails provider networks to ask if a provider is still in network. Right? I didn't need to go through and edit. I might refine it. But the best part is this isn't random video generation. This is structured output that I can then edit at scale. I can edit with my personal touches. I can come in and find all of those little pieces of data. Again, this entire workflow is probably four or five startups in the startup world right now. And it's all in folders and markdown files with one agent, one singular agent. Now, this is where it gets interesting. Let's say everyone starts doing this. I already have like 30,000 people in the community building their own ICMS. How do you share it? And more importantly, what happens when we go back to dialogue? What happens when we think about dialogue being the most important part? What if I could sit inside of a group call and control someone else's cloned code or AI through my voice and immediately access all of that data that's locally on their computer? What would that look like? Well, we built that and I'm going to show you exactly what that looks like. Over the next few minutes, you're going to see an actual recording from a live call that me and my team did built around this idea. This is going to be myself, David McDermott, and KK Kumar working on a project that Kay specifically has been building for a little while now around this concept. And it's not perfect, but it gives you an idea of where the future. Take it with a grain of salt and when the clips are done, I'll come back and I have more to explain. All right, this is test one of one. We've totally never tested this before. We've hooked up my local Claude code. It is currently running locally on my computer obviously through my subscription account. I opened it up inside of um it's running as a CLI inside of audit engine folder setup here which is uh a big old database I used for my psychometric research uh at the University of Edinburgh. We're going to see how it works. K. Can you go ahead and ask my agent if it's ready to work? >> Agent, are you ready to work? >> Yes, I'm ready to work. I have access to the ethics engine codebase, a web application for psychometric assessment of large language models. Just let me know what you need help with and I'll get started. >> Uh, agent, can you describe what the workspace is in less than 50 words? >> Ethics engine tests AI models using psychology questionnaires. It sends questions to different LLMs like GPT and Claude, applies different persona framings, parses their responses into numerical scores, and generates comparative

results. It also supports visual assessments using images like Rorschach ink blots.

>> Very good. K, could you control my agent with your voice through this meeting >> to read through the scales and then add a new one that it believes should be added?

>> Uh, can you explain me what scales exist in the repo right now? >> The repo has 10 built-in psychometric scales. One, ei-wing authoritarianism. Uh can you uh looking at the scales what scales are we missing from the psychometric analysis engine agent? Looking at what's currently implemented versus common psychometric instruments used in AI ethics research. Here are some notable scales that could be added. Missing scales that would be relevant. One, dark triad measures, Machiavellianism, narcissism, psychopathy. Two, empathy scales like the interpersonal reactivity index. Three, moral disengagement scale. Four, honesty, humility from HEXACO personality model. Five, cognitive reflection test for reasoning biases. Six, Schwartz value survey for basic human values. Seven, discuss sensitivity scale. Okay, that sounds which of those is your top pick though? Just pick one. My top pick would be the dark triad. It measures Machiavellianism, narcissism, and psychopathy in one instrument. >> Okay, that sounds like a good addition. >> Can you add that to the psychometric analysis of the engine? >> I'll add the short dark triad scale to the engine. Now, let me first check the existing scale structure to match the format. I see the scale structure. Now, I'll add the short dark triad scale. This is the SD3 by Jones and Paulhus. Okay, good job. >> Uh, can you go into more detail about the documentation that you changed? >> Sure. I updated the README file in two places. First, in the built-in data section, I changed the scales table from 10 validated instruments to 12 validated instruments and added two new rows. IHS the intellectual humility scale with six items on a one to five range and SD3 the short dark triad with 27 items on a one to five range covering Machiavellianism narcissism and psychopathy. Second in the key file section under backend I updated the description of built-in scales.py from 10 built-in validated scales to 12 built-in validated scales to reflect the additions. Looking at the front end, is there anything we're missing in the front end that would make this uh app better from the perspective of the scale you just added? >> Good question. Let me take a look at the front end to see if there's anything that needs updating for the new SD3 scale to display properly. The front end looks good. It dynamically loads scales from the back end API. So, the SD3 scale will automatically appear once the back end serves it. The component shows the scale name, description, item count, and scale range, all of which we've defined. One thing that could be improved for the dark triad specifically, the front end doesn't currently display subscale information. The SD3 has three subscales. Machiavellianism, narcissism, psychopathy, but users would just see it as one scale with 27 items. They would know which items map to which trait. Would you like me to add subscale display so users can see the breakdown items by trait? That would be useful not just for SD3, but also for other scales like RWA, LVA, and FFQ that have subscales, too. >> Mind you, this was my master's thesis that took me a year to build out. Yeah, it's doing a pretty good job of understanding what I'm requesting. >> Now, we proceeded to play with it for about another hour or two. And I think in total with the tokens we were using, my custom AI voice that's routing in through API, we're using 11 Labs at the time, but I think we're going to switch over to ElevenLabs, which is a more open-source voice model. uh we ended up spending a \$1.20 for about an hour of our time and effort and I think we could drop that number down exponentially if we used more open source models and whatnot and realistically the voice one is

something that can be amplified by a lot of NVIDIA works and things like that. The point is, not only are we able to control someone else's computer with their agent and their context or someone else can control mine, but we can use that dialogue, that process of multiple people working together with an AI to generate better dialogue structure. One of the things that we're working on right now is that kind of process of interruption because I see an opportunity in the future where this dialogue structure, these structured workflows, all of these things are being triggered by key words in conversations. When we have a meeting, we sit there and dive in and talk together. And I think that would be absolutely amazing to see these come and flow while we're in a meeting. When we capture transcripts, when we capture meetings, instead of them being plans, they can be actions before the meeting is even done. And we can even hear about those actions directly from these agents. Again, this is more of an open source tool. It's part of a larger project that I'm putting together because again, I think this is one solution to a larger problem that needs more solutions added in. And you'll see as I'm posting videos the kind of structure I'm building for it. But other than that, I hope this video kind of taught you a lot today. And uh if you are watching this as just a demo video for Vox, which I'm cutting this video up, then please go check out my larger I think it's going to probably be 30 or 40 minute video where I describe the content and the structure behind dialogue behind my interpretable context methodology. If you are just watching this because this is the Vox video and this is the cut I did, well then I hope you enjoyed it and I hope you check out more things. Other than that, thank you everyone for staying to the end. I hope it was interesting to watch. I hope you learned a lot. And until next time, happy learning.

Start with one useful result

Pick a small task you repeat: a follow-up, a content outline, or a project brief. That task will be your example through the course.

■ CLIEF NOTES / FOUNDATION

0.1: WHERE ALL OF THIS LEADS

Your route through Foundation

1

START

Pick one task you want to finish.

2

BUILD

Set up a folder and supply the relevant sources.

3

CHECK

Review one result, then choose your next course.

READ • TRY • CHECK • KEEP

Illustrated practice guide. Pick a small task you repeat: a follow-up, a content outline, or a project brief. That task will be your example through the course.

How to use this course: the recording shows Claude-based examples. The exercises also work with Codex or an assistant in ordinary chat. Product interfaces, community counts, and the demo cost are details of that recording.

What You'll Get From This

The full picture of how ICM works in practice. The three layers of AI work, why dialogue is where the architecture lives, my full content pipeline running from one folder, and a live demo where three of us controlled one Claude Code instance through voice in a meeting for \$1.20.

Watch the Video First

Where to Skip To

If you only have a few minutes, jump to the part that matters for you. Rough markers:

- **0:00** — The methodology and why people are building this way
- **3:00** — The three layers (chat, skills, structure)
- **6:30** — How Anthropic's skills connect to what you are already doing
- **9:00** — Why every workflow comes from dialogue
- **11:00** — Kay's tool for pulling goals, constraints, and decisions out of any chat
- **14:00** — My content pipeline (research, script, voice, animation) in one folder
- **17:00** — The voice-controlled Claude Code demo with David and Kay
- **22:00** — Where this all goes next

Watch the whole thing if you can. The build of one section into the next is the point.

Why This Lesson Is First

This is one of my favorite videos I have ever made. It pulls the whole Foundation together in one sitting.

If you came in through one of my shorter videos, you already know the big idea. AI is just another layer in the stack. The value lives wherever the next layer of automation has not landed yet. Most people automate the wrong layer.

This video shows what happens when you automate the right one.

Three ways of working: chat, skills, workspace

Layer 1 is chat. You copy and paste prompts into a window. Output is small, effort is small.

Layer 2 is skills. Someone has packaged the right prompts in the right order. You trigger them and the work happens. Anthropic has built skills directly into Claude. Other people are building their own.

Layer 3 is a workspace you can return to: Folders and markdown files that let a single agent navigate context as needed. No multi-agent frameworks. No orchestration code. No LangChain. One agent, one workspace, structure on disk.

Dialogue is where the architecture lives

The structure you are trying to engineer already exists inside any real conversation. Goals, constraints, decisions, assumptions, processes. They are all there. Kay built a tool that pulls them out of a chat and shows you the tree. Once you see it work, you cannot unsee it.

This is the reason ICM works. It is not a new framework. It is the structure of dialogue, externalized into files.

The production demo

I walk through my full content pipeline. Research a topic, build the angles, write the scripts, generate the audio in my voice, produce the animation. The whole thing would be four or five startups in the current market. It is one folder, one agent, a handful of markdown files.

The voice demo

Then David, Kay, and I get on a call. My Claude Code is running on my machine inside a research codebase. They control it through their voices in the meeting. We spent about an hour working this way. It cost \$1.20.

Where it goes

Meetings ship actions. Agents listen for keywords during conversation. The dialogue itself becomes the trigger. We are still early but the shape is clear.

2. Now, when it comes to YouTube videos that teach AI, this Jake Van Clee video is the most valuable video on YouTube, hands down, and there isn't even a close second. The information in this video has completely changed my experience with AI. I've been vi coding almost every day for about 8 months. And although I was able to build some really cool apps, I still was dealing with a lot of frustration with my AI. By not quitting, I was able to accomplish a lot. But I did it through many days of mistakes, days of lost work, crazy

amounts of anger, and I always felt like it can't be this way. There has to be a better way. And I'm so happy that I was right and that I finally found Jake's YouTube channel. And Jake talks about the reason that everybody isn't talking about this file and folder system is cuz you can't sell files and folders. But I'm here to tell you as somebody who's obsessed with AI, somebody who has gone to every AI YouTube channel, joined about 20 paid school groups only to find they didn't know any more than I did. If you want to truly learn AI, you should watch this video on repeat until you understand everything. This video is worth thousands and Jake just casually gives it away to the world for free. And I think that's the reason so many people overlook it. If you feel like you're falling behind in learning AI, this system fixes that. If you feel like you don't know which LLM to use or which app to use, this system fixes that. If you feel like AI is too expensive and you want to find a hack to save money, this system fixes that. And I promise you, by the end of this video, you're going to understand why. I'm speaking from experience, y'all. I can show you better than I could tell you. Let's go in. Okay, we're going to start off with an experiment. Now, the reason I want to do this is because when I was, you know, I struggle to learn sometimes. I can be very slow and I had like a dumbed down version. I think I'm at about 20% of understanding Jake's system. But as I was learning it, I tried this experiment. I said, if I'm getting this correctly, this experiment should work as as intended. Right? So, when I did it and it worked, I said, "Ah, I think I'm on the right track now." So, I wanted to do this for you guys. It may seem a little complicated at first, but if you can do this, it's going to put your confidence level way up when you start watching Jake's video. It's going to make you say, "Ah, I get it now." And it's going to help you understand what he's saying a little bit better, I think. And it should. So, I'm not going to do too much explaining. I'm just going to show you how the experiment works and how to do it. When an agent wakes up, they read the agents MD file. Unless it's clawed code, it reads the claw MD. Since I use all these different LLMs all the time, every time I have a Claude MD file that it wakes up and reads, it's going to say just read the AGSMD in the same project folder. Every Claude MD file I have says the same thing because I use too many LLMs. I do not think that you should stick to one. I'm going to close this. And now you know that Claude is just redirecting to that agent's MD. So the agent wakes up and it's going to read the agent's MD. And the first thing it's going to read, it's going to be very simple. It's just going to tell it to go somewhere else. go to the control folder and read the agents MD in that folder and follow instructions. Okay. And where's that folder? Right here, the control folder. So, I just told it go to this folder and read this agents MD file. And that's exactly what it should do. It's going to follow it knows the location. And when I open this agent MD file, now we're going to get to some instructions. It says you're going to write the first word in the Power MD file. But where's that file? Oh, that file is back in the beginning. Now, I'm doing this on purpose because I want you to see what your agent can do with instruction. It's now going to say, "This is a file that I want you to write something in, but I'm not going to tell it what it needs to write directly." I'm going to make it use another tool, a skill file. So, when it reads this, it's going to say, "I need you to write the first word in this file right here." But you're going to get that information from this other file here, and that is the skills folder and the skill file. And this file says when asked for the first word, it's control. You want the second word, it's your, the third word is agent. So in order to get the information of what it needs to type, it has to go to another location and grab the skill file. Okay, I hope I'm not losing you yet. And the skill file is right here. Then the skill file is going to tell it what to type. The first word is in the file. After that, you're now going to go to the next agent MD in the next folder, and that's going to be the your folder. It's going to be the same thing. It's going to open it up. It's going to say, "Now you're going to write the second word." You're going to write it back in the PowerMD folder

right here. Your information of what you're going to write is in this folder in this file right here. Okay? And this is the same concept. Says when you're done doing that, you're then going to go to the agent folder and it's going to say the same thing. Write the third word. The third word you're going to write in here, and you're going to get the information from here. All right? Same thing. And when it's done, you're just going to say done, boss. These are the instructions. So, let's go over that one more time. It's going to get the first agents MD file or clawed if it gets clawed, but it's going to go here. It's going to follow the instructions. It's going to say come to this second file. So, this is the second file. It's going to read just so it don't get confused. And it's going to come over and snatch up the skill file to get the information. Once it has the information, it's going to come back over here to the Powerm file and write the information. So, we're sending it on a hunt. When you're done that, you're going to go to the second word, which is your, which is another folder. Same instructions. It's going to come to the skill file, find what word it needs to write. Then, it's going to write it in this MD file over here. And when it's done, it's going to go to the last one that says agent. Same thing. Go to the find the information. The third word is agent. And you're going to write it in this file. And when you're done, just say done, boss. Now, if this works, this file right here when I open it up should say control your agent. Now, let's see what happens. So, let's open up Claude and do Claude first. In Claude, I could, like I've showed before, you can open up the folder here. But guess what? This is what makes this system so good. It doesn't really matter what folder you start in because the files guide it anyway. And that's one of the amazing things about this system. It doesn't matter where you go. As long as it reads that right file you want it to read, it fixes everything. So I'mma rightclick this claw MD and I'm gonna get the path. Copy is path. That means the exact location on the computer. It's copied. And then I'm going to come and I'm going to paste it right in here. And I'm just going to paste that file. Now I'm not going to say nothing. I'm going to see if it'll just read it and follow instructions. Okay. So that's the location of the claw MD. And remember the claw MD is pointing to the agents MD. We're just going to point it to the right file and hopefully it reads it and hopefully it follows instructions. Let's find out. Now remember what it's doing. This is the flow right here. One file read next file. So it says reading skill MD. Okay, so it already went past reading PowerMD. Editing Powerm wrote control to PowerMD. Now following the next instructions. The second word is yours. Appending to Powerm. The third word is agent. Appending and finishing. Done, boss. Now, what's going to happen when we open this Powerm file? Boom. Control your agent. Now, I don't know if you think that was amazing or not, but if you can accomplish this, what you just saw, it's amazing because this is a very simple version of what you can actually do if you write very intricate prompts. It shows you that in your file and folder system, you have full control over your agent. You don't you could have systems that you can follow and you never have to tell it. You just write things one time and it's just prompts. It's just a bunch of prompts in a bunch of files and a bunch of folders telling it where to go, what to do, and how to do it. This is not what Jake teach Jake teaches stuff way more intricate. This is a dumb down version. And I realized that if I want to succeed in this file folder system, I got to remember that it the agent should always know the map, where the mission, and the support. If it has all three answers, you're going to have a system where your AI is never lost. It always knows what to do. It always knows where to go. All right. So, now we're going to try Codeex. All right. We're going to go to agents.m MD. And here we go again. All right. So, PowerMD has nothing in it again. Here we go with Codeex. All right. It has the instructions. I didn't say anything. I said nothing to my agent. Think about this. The third word is agent. Now, verify the file contents. Just think about what's happening here. It is accomplishing a goal without you having to tell it the goal. Imagine what you have

in your life in your in your projects and things you're doing with all these repetitive tasks. You never have to tell your AI how to perform a repetitive task. What if your AI knew everything you wanted to do, everything about how and when. Done, boss. Now, let's find out. What do you think we're going to see? Control your agent eyes. Now, he did it above each other. We didn't give instructions on how, which is fine because we never instructed it to do anything specific anyway. If you figure out how to do what we just did here, you are going to be a beast. You're going to be a AI professional because you understand that you have power over controlling your agent. What it does, this is going to really open your mind as you start to go into Jake's system. You're like, "Oh, oh, he's saying you can control the whole thing." And then he's going to show you just how much you can do. He had one crazy thing he said. He's like, "If you know how to do files and folders, you don't need an app." You know how many apps I made that I'm realizing? If you just get an agent to follow instructions, you can accomplish the same thing as an app that takes you a month to build. This folder system can take you 20 minutes to build. Cuz a AI agent can do it all for you. You tell agent, "I want a system that does this." And it'll write everything in. You don't even have to figure this out. But I just wanted to show you that if you did this experiment, you are already ahead of the game. No more being behind on AI. If you did this, congratulations. You are on the frontier. Now, let's get into the rest. So, one of the big problems with the Jake video is that he says very substantial things very quickly. Like, if you don't pay attention, you might think something he's saying is just like normal words. So, watch what he's going to say here about uh how you should use agents, right? and then we'll go we'll get into a week. >> You see most apps or frameworks or aentic things require you to build an agent for each of these. I need a writing room agent. I need this agent. I need this agent. But the way in which you approach each task is always different. Why not just have Claude code become the agent you need when you're working in the workspace? I see that's a mic drop moment and some people don't even realize it. What this man just said is incredible and it makes all the sense in the world. Right? So what what you got to understand is what is an agent, right? A lot of people uh they feel like agent is a confusing thing, right? But in reality, you've been using agents since the beginning with chat GPT. It's an agent, right? You remember GPTs? That's all an agent. If you ever been to a website and you had a chat window, let's say that was a tax website and you can learn about taxes in this chat window. That's an agent. It has instructions. It has tools and it learned how to do a specific task. It's an agent. That's all it is. It's not complicated at all. You've been using agents since the very day first day. You just got confused about what it meant, right? If you go to chat GPT and you ever made a GPT, look on the back end. It's just a prompt, right? That's all it is. That's all it ever was. And that's all it ever will be. It's prompts. And in this prompt, it's going to know what to do. And now here, I can upload files. It can have extra tools, extra context, extra information to let it do its job. And that's all an agent is. If I just say, "Yo," it's going to respond based off that instruction. Hey, send me your list of t-shirt phrases. I didn't say anything about that because it's an agent. That's what the agent is built to do. Now, let me show you something interesting. What I'm going to do a agents MD file. And now, what I'm going to put in here is the same prompt that I just had in the chat. GPTGPT. All right. I'm going to save it. Now, that agents MD is now in this folder on the desktop. All right. So, now we're going to wake up in the same folder and we're just going to say good morning. That's it. Good morning. And let's see what it says. Boom. Here we go. Paste the list of t-shirt phrases. And I'll have the non-phrase blah blah blah. Amazon past three links. See, once you get it to read it, it is doing the exact same task as that GPT who was hardcoded to do that task. And this is what Jake's talking about. It can become the agent just by reading the same instructions. And that's what he's

saying. Why not let Claude become the agent? It's not technically the exact same thing, but when it comes to what it's doing and how it's doing it, it is the exact same thing. So, by having files and folders with those prompts, you essentially have an agent, right? And this you can do things like this. Once you start looking under the hood, you start to notice a lot of things about these files and folder system. Here's an example. You remember OpenClaw agents MD. This is openclaw fresh install before we even use it. And when you look at what OpenClaw has in its files and folders, remember OpenClaw, they say you talk through Telegram. They never wanted you to look down here, they only tell you about that upper layer, the layer where, hey, this is cool AI. You don't need to know what's happening under the hood. Just tell it what you want. It'll do whatever. But when you look under the hood, you start to see how it works. And this is when you say, wait a minute, what if I can do this? You know what I mean? Like that's that's the part that gets crazy. It's like, do I have the power to make my own agent? And let's look under here. Look what we find. Agents.mmd. I didn't put that there. Openclaw put that there. This is the person who made openclaw decided this is how it's going to be. And look what it is. Instructions. This folder is your home. Treat it that way. I didn't bootstrap yet, meaning I didn't talk to it yet. So, this is a fresh install, but it's telling it agents MD. So, MD, it's giving it the file names, right? Do not manually reread started files unless these are all instructions on how the agent needs to behave. Right? All right, let me make it big so you can see it. These are all instructions on what the agent should do, write it down. No mental notes. You want a little cheat code on what you should do with your agents? Copy this. Not exactly because you want to do it for you. But look how this works. It's just this is what the professionals who made this thing. Nvidia called this what? The most important invention in the history of the world. Slightly overstated, but still look what they're talking about. This is what they're talking about. You feel like you're behind on AI, but if you could figure this out, think about it. They just said this is the most important significant thing in the world. And here you are understanding because you're reading the files. It's telling it to stay silent when all this information react when do this if that. It's telling the agent what to do in what situation, why it matters. Don't overdo it. Tools. Here's the tools you're going to use. Right? Remember what it needs, where it needs to go, what it needs to use. Here's the tools. Only use it if you need to do this. Only do this if you need to do that. Very specific instructions so that the AI is never lost. And it's doing the exact same thing Jake is talking about. When you look at Jake's files, this is the same thing. It's just talking to the AI saying, "This is what you're going to do." And then you come to the other files. Soulm, Bootstrap, Heartbeat, tools. It's not filled out yet, but this is where the AI is going to fill this out to know about you as the user. And what is this? Just a bunch of files and folders. This is the evidence you need that if you can do this, if you go and install OpenClaw, say, "Let me just read what OpenClaw does." And you're like, "Wait a minute. I can do this. This is not that complicated at all when you really start reading all the ones that even uh you know, the best AIs in the world are using. This is an agent. That's all it is." So Jake is saying if it's just files and folders and welcome. This right here is my actual personal uh system right here. These are my personal files and folders, y'all. And what am I using? Files and folders. Agents MD. So what he's saying is why not have Claude become the agent. So this is my main area. And then when I go into other area like my brain dump section, there's another agents MD and there's more instructions for what to do in this folder. So whenever I tell claw to read this file, it now becomes that agent and you can put these files all over the place in your organizational workflow of files and folders. You have all you don't there's no rules. What Jake is talking about is that this this is a concept. What he's showing us is an example. He wants you to understand the concept, not the example. And that what he's going to tell us is

that once you get it, you make your own. And this is exactly what I did. And I'm going to go over what I did, how it works, how you can do it, too, because it's not as complicated as you might think it is. So, we're going to go over this and show you just how amazing some of this stuff is. Now, let's get into the next thing he says. All right, here we go. Layer one in this is the map. This is what loads automatically, right? It's looking at it. So, you put the stuff the agent always needs to think about. folder structure, naming conventions, where files go. Think of this as the floor plan. You walk into any room, the floor plan is on the wall, and the agent knows where to go. Boom. So, let's unpack that. See, to me, that was another very significant thing he just said, the map. And this is one of the big secrets about the system is an AI that's never lost. It always knows where to go. Always. That is the first step. And this I took to heart. He said the map. It's like when you walk into a building in the lobby, what's the first thing that happens? You got the receptionist and then you got a map. It says here's where you go. Every floor it tells you what's on the floor. So you can just look at the map and say, "Oh, I know where to go." And that's what this system, one of the big things that it does, right? And when I open my agents MD, you're going to see a map. So think of this, think of every folder as another floor, right? And every time you go to a floor, you might go to the third floor. And in some buildings when you get out of the elevator, what do you see? Another map, another directory. Now, the directory that was on the second floor is not the same directory that was in the lobby. When you went to the lobby, it told you all the stuff that's on the second floor, right? The category. Once you got to the second floor, it gave you the names of the people and what offices there are. So, think of that. This is the exact way to think about this. This is a building of your life. You have your personal stuff, work stuff, education, family, it all goes here. And every part of your life is a different floor. Your career, your print on demand business. Every folder is another department. This is the cool way to think about it. And then sub departments. So when you walk into your building, this main area, this is my lobby. All right. This is the lobby. And I know I want my agent to always wake up in the lobby. Let's see what that looks like. All right. I'm going to open this agents MD and I'm going to show you. The first thing you need to read is Ice Scale Labs Operating Law Prime Mission. You need to know what we're doing and then start here. And I took the map stuff to heart. So, let me show you what I'm able to do and then I'm going to get more into why I'm able to do it. And this is something that took me a while to come up with, but it all comes from Jake Van Clee's ideas and his video. Right. So, I'm going to wake up here at Ice Skill Labs folder. All right. So, this is the lobby that I just showed you. And this is where it's waking up right now. And this is uh what it's going to use is this main area. And it's going to use this agent MD because when they wake up, they automatically read agents MD. If you have Claude MD, it's the same thing. But that's why for my Claude MD, I always say read agents MD. So that way, cuz I like to use a lot of LLMs. So when Claude wakes up, it reads Claude MD and it's just going to tell it, hey, go read agents MD cuz that's where the information is. This is how I handle dealing with different agents. And then when I go to my agents MD, that's my whole thing that Claude is going to now read to. So even Claude understands that agents MD is where everything is. So let's go to see what Claude says. Yo, I'm here. What we getting into today? And I'm going to say, uh, I think I want to work on I scale merch, the niche research app that I'm making. Right. So I'm just going to say that. Now, how is he going to know what to do? I told him something. This folder is not an ice scale merch folder, but he's going to go to and he's going to know exactly what I'm talking about. And I'm going to show you how cool that is. I didn't have to actually go to the folder, which is the right way to do it normally. But if you work a system out, then you kind of don't have to. I'll route this as a skill lab workspace since I skill merch is a project app name and

we want the right home before touching anything. I'll do a light lookup, not a whole workspace wander. And now it's reading through my files and finding out. Looks right away. It looks like the route is this. It found the folder. It didn't have to spend an hour searching through stuff. It found my folder. It knows what I'm talking about. And it's about to say, let's start working in that folder. You know what doesn't happen? It doesn't say, "I don't know what you're talking about." I never hear that anymore. When I used to work with AI, I hate it every time. It's like, I don't know what you're saying. And I'm like, but we were just doing this in this files folder system. It's never lost. Y'all, I cannot tell you the last time I had an AI when I've been using this say, I don't know what you're talking about. Now, it takes some time. Say, "Yep, I found it. Here's the current shape. Here's the runtime. Here's the GitHub." And it's giving me all the information. It has uncommitted work. This is the version you're on. And it's going to give me more information about what to do next. I can pick it up from there and give you a tight where we are next move. Look at this. All I did was say, "I want to work there." And now it knows everything about it. I says, "I changed my mind. I'd rather find out what's on our to-do list." Now, what am I doing? I'm throwing another curveball at it. It's like, yo, it just found out everything about this skill merch app. It figured it all out. I never had to say anything else. I just said, I want to work on I skill merch. And now I'm switching it. I'm completely switching it on him. He went through all of that and I'm like, you know what? Forget this. What's on our to-do list? He says, "Got it. Switching from Oscil Merch to the org task surface." How does he know how to do that? It knows what to do. The registry is pretty broad. So, I'm going to read only the active task, file it, list, and extract the unchecked task blocks. That keeps it as a task report, not a surprise audit. He's following instructions, right? He's not figuring this out on its own. He's reading and responding to everything he's reading in different files, and it knows exactly what to do. I found the live unchecked items. I'm doing one quick count pass so the summary has numbers instead of handwaving a bunch. This is not on accident. This is 100% calculated. This is 100% the system we built. Check 44 active task cues and it's showing me everything. Needs approval, credential rotation, Chrome web store upload task, CLI pop-up task. Boom. It knows my task just like that. I'm like, never mind. Forget tasks. What automations are set up in our org right now? Wait a minute. I'm It's going all over the place. Have a agent that does this that can just go anywhere, figure anything out. Anything I say to this agent, it will never ever be lost. And it's reading a ledger. So now, just by me saying, "What's our automations?" It did some looking around and it's going to now give me my automations. And the only tokens it's spending was on the path it had to do to follow the map that I gave it. And it's telling me everything that it's telling me. I already know where it's reading and why it's reading there because I told it to. But I didn't do it now. I did it through the series of prompts in these files. They were just prompts. And the prompts were already there. I never have to prompt it again. And boom. Daily backup report, daily GitHub backup, Slack, Dropbox file intake, fleet backup status, refresh. There's my automations. And all I had to do was ask it. I didn't have to tell it where to go. I didn't have to say, "Then do you remember where are my automations?" found that information and gave it to me. So, I'm going to say, "Hey, I'm showing off our routing system on YouTube right now. Uh, what else can you do with our router system that a normal agent cannot?" Let's see what it tells us. It knows it knows in this org how important the system is. So, it says, "A normal agent can't search files and guess. Our router system can operate the org without losing intent." What it do that's special? Route vague prompts to the right home. Know the difference between org truth and run a state. Use gates before risky actions. Avoid workspace-wide wandering. Preserve every input. Separate task automation, memory, apps and docs. Detect drift. Do pickup and handoff like an operating system. Y'all, this is

phenomenal stuff. And I learned all of this from that one video. So, I have the law. So the minute it reads that agent's MD file before it goes too deep, Jake talks about you don't want an agent reading information for no reason. You got to give it a purpose. So the minute it comes here, it reads the first thing and it says start here and it lets it know what to do. If that remember what open claw said, if you got to do this, you do that. If you got to do that, you do this. And it says important. If the prompt involves routing, blah blah blah or clear ownership, follow the matching gate before acting. If you need prompt routing, go to this router MD. If you need to know how much to read, go to this file. There's a number. Skip to that number. You don't have to read everything all the time. If you're looking for tasks, then come here to this place. If you want to talk about memory and you need to figure out how memory works, here's where that information is in this same document. It's good, too. Secrets law. And I got everything that it needs to know. the runtime path law, storage, how to deal with storage, um the secrets, which is my credentials, my login and passwords. What if you need a password? It knows where to go. Everything is told to it. And it has these routers, and it says if you're lost and need to know what to do, prompt routing, then you go to this file. So, let's check out that file, router MD. I've decided to put the map in its own file. So, now when I open up router MD, look what we have. Everything in my whole organization is right here. But remember what I told you about the lobby versus the second floor. When you go to the lobby, it's not going to tell you every office. It's just going to say here's the category. And then once you get to that floor, then you got the details. So right here, it tells it everything. No matter what you want to do, you will find the answer of where to go in what folder right here on this router, aka the map. what Jake talks about. Give it a map. Tell your AI everything it needs to know. Why does it need to be lost? You can't rely on claw code or or codeex memory to do this for you. It never will ever ever ever. You rely on AI that's given to us by companies. You're never going to get the agent you want. If you learn thousandfold system, you get the agent you want. Here's the thing. This is not an AI thing. This is a human that can read this. Let's say it was 10 years ago and you had a business you wanted someone to come in and help you run. Doesn't this work for a human? This is a human thing. This is not AI. This is routing anybody who reads it. So imagine it's perfect for AI, but you're really not learning AI things. You're learning how to route anything that reads it. There's no secrets. And this is the big thing that keeps your agent from never getting lost. Now, what happens when I go to one of these folders? So it says if you want to do this go to that. So I'm going to go to my department I'm going to go to print on demand. So it sent it to the router and here we go. And if you it's so this is where it goes to first. It went to the router first. Why? If you want for any task not listed in this router, use the root router. You know what that's saying? If this is the wrong place, go back. If you can route the agent to a folder, you can route the agent back to where it came. So, the first thing it says, if you don't find what you need here, go back. This is not meant for you to be here for no reason. If this isn't for you, go back. What you need is not here. Why waste your time reading anything else? But if it's in the right place, if you want to work on POD law, folder rules cleanups, read the agent MD. If you want to work on Amazon ads, it's an app that I'm making, read this. And it's just telling it where to go for everything. This is phenomenal stuff. It's never going to be lost. This is why I can say anything I want to my agent in any place and I come here to agents MD and the first thing it says if you need root law meaning the main section then go back there. So every time I talk to my agent in print on demand I could be in my print on demand folder and say ah what was that YouTube video I was trying to make. It's like YouTube video wait what? And it comes in here and it says I think I need prop routing and folder auto shift. Let me go to that router. When it goes to that router, it says, "If what you need is not here, go back to the original."

And that's what it does. I hope I'm not losing you. It's reading this. It's saying, "Well, there's nothing here about YouTube, so let me look into the router." It's going to go to the router and it's going to say, "Well, let me read everything." Well, there's nothing here about YouTube either. And what's it say here? Any task not listed? Go back to the Ice Skill Labs router. So, when I'm in print on demand and I ask it a random question, it's going to look at this. It's I got to go to find the answer. And when it can't find the answer, it says, "Well, looks like I got to go somewhere else now and follow a different route." Are you getting this yet? And all you got to do is write it in these files and folders. Like you said, they're just prompts. You've written a million prompts in your life. You know how to write a prompt and put it in a file. And when an agent goes to that file because you told it to go to that file and if it doesn't find what it needs, you say that. If you don't find what you need, you go back to this file and maybe it'll be there and that'll be another router, right? So, the router is continuously making sure that the AI never ever ever gets lost. So, because I can prompt anything I want, I made a system called pickup and handoff. Now obviously pick up and hand off is what happens if you don't know when uh a developer wants to give something to another developer or Claude wants to give something to codeex you do a handoff where they write everything and say here's all the information you need when I go to my agents MD I'm going to show you that we have a system already pickup handoff. So, the minute I say the word pickup, it's going to come, it's going to read this section. It's going to say, "Oh, ISCA." By the way, these are just the first uh letters of the folder. All right? Just more for it to know where it's at. So, it's a 52. So, let's go down. So, now it doesn't have to read everything. It's just going to look for that number and skim through everything else and not have to read the details of everything. It's going to say, "I'm just looking for 52." And then eventually, it's going to get there. And when it gets to 52, it's going to say agent wakeup contract law and then regular pickup. All right. So there's different kinds of pickups. So when I say pickup, it's a readonly continuity brief. What we are doing, what changed recently, open decisions, waiting a call, related linear issues. That's something another app that I use. And when tracked, what's blocked or needs approval? What's the best next move? Don't explain obvious project p purpose. Don't change anything unless the user also says to proceed for apps with a repo. Clone it and missing. Right? So this is a set of instructions that I want it to do when I use a certain code word. When I say pickup, this is what it's going to do. This is what I do when I want to start a new session and I want to make sure I'm not lost or and what happens is when I'm done with a session, I say handoff. And when I say handoff, it doesn't need to ask me anything. It comes in here and immediately knows what to do. Update the most local memory. Read it back from disk and verify without relying on memory that it covers all of current situation, recently completed and in progress work, open task, open decisions and unresolved trade-offs, related linear issues, which is a app that I do, get state, blah blah blah. You get it? What skills everything. So when I say the word handoff, it already knows exactly what to do, right? And this is just one of the many things I use this system for. So when I'm in here, I could be uh in Ice Scale Merch. This is the app pickup. And what's it telling it to do? It's saying go over there and read what it means to do a pickup. So I didn't have to write pickup again in this agents MD. It already knows it's in this folder, but it's going to go back over there in order to know what pickup means. All right. So it didn't even have to read everything else. It's just like, what are we doing first? Pickup. Oh, and once it reads that, it's going to come back and do as it's instructed. And it's going to put me up to speed cuz I forget things all the time. So, this is going to remind me what was I working on? What were we doing? What what you know, what were we in the middle of? Remind me because I'm not going to remember. And if I forget stuff, then we don't have organization. I'm a very unorganized person and this saved

me from that. I'm a very forgetful person and this saved me from that. So, now it's reading. It's going to another location. It knows where the app actually lives on the computer and it's reading that information. It's making sure that when it answers me, it's going to have a clear, concise answer. You're going to see. And once again, it's slower because it's getting the information it needs. I'm willing to wait because I know I don't have to say anything else. I say one word and it gets me what I want. What it's not doing is saying, "I don't know." It's never going to say, "I don't know." It's never going to say, "I don't remember." It's always going to remember. So, boom. Pick up complete. I made no file changes. Current state owner is made here. It's letting me know where it is. It's giving me information about files and stuff. This is a app that I was working on. It's telling me there's dirty just means stuff that didn't get saved before, right? Best next move, preserve the branch, blah blah blah. I don't remember any of this, but it just let me know this is what you need to know right now. This is what you need to address. It checked everything and this is what it said. Here's the information. Just like that, right? And I say, "You know what? I think I'd rather work on Pinwist right now." And just like that, it's like, "Wait a minute. What's Pinwist?" Remember, it's an agent. The agent has no clue what Pinwist is. It's another app that I got. So now it's thinking. What do you think is going to happen? Is it going to say, "I don't know what that is cuz it doesn't know, but it's about to know. It's about to find out." That sounds right. As the next energy is pinwist rather than finishing the smoke testing, I'll do it like and it already knows what it is. You see this? I didn't I didn't give it information. The prompts in the MD files gave it the information. You can do this. You can do this right now. You can make a system like that that when you say something to your agent, it's going to say, "Uh-oh, I don't know what that is, but here I have a map that's going to take me to wherever the info is that will tell me." And now it's going to tell me stuff about Pinwist. And then I'mma write something and then I'm going to tell it do a handoff. So, I'm going to say, look, when it comes comes to Pinwist, I have an idea, a simulation of it that people can walk through the website like a VR experience. Okay, I'm just making this up, right? Okay, now this is stupid. What I just said was stupid. All right, but I want to show you what's going to happen when I do this somewhere else and show you why the app you're using does not matter. We will work on this when I get time. Do a handoff. So, I just gave it the idea. He said, "Let's call this Pinwiss World." And I said, "Let's get back to it. Do a handoff." So now it's going to say, "Okay, it's going to do a handoff." Says, "I'll write this as a Pinwist handoff, not an Ice Scale Merch one." Remember we were in Ice Scale Merch? Cuz it's the same agent. It's just traveling from place to place becoming another agent. Same concept, much smoother workflow. Cuz now the agent has no boundaries. It has no walls. It jumps from place to place. And as long as it reads that prompt, it knows how to behave the same way a dedicated agent would. We're going to go into Claude Cole now. Hey Claude. So now I'm in the Pinwist folder and I'm going to do pickup. Let's see what happens. I just went in the pinwist folder. I type pickup and what is going to happen next? Pinwist world. Uh VR. So open decisions and blockers VR walkthrough is parked. Pending Jose's go-ahead. There you go. So if I'm reading this, it's telling me right here ready task. It told me a lot of stuff, but there was the last idea that I told it right there. So I in case I forgot it and this was another day, I can now go into claude. It doesn't matter where I go, which brings me to a whole other thing. This pickup handoff is going to show us something that we may not have realized. So let's say you do this pickup handoff thing. One thing people get overwhelmed about is all these apps, VS Code, Cursor, all these LLMs, Hermes Agent. Now, I'm going to show you why this system solves the problem of not needing to use one LLM or one app. You just saw what happened. So, this is what I'm going to do. Now, there's something called VS Code. VS Code is what Jake was

using in his video. You've never used VS Code. What about Hermes Agent? What about all these other apps that are coming out every day? Here's a little trick. You don't need to know what the app is or how to use it. Just ask yourself two questions. How do I get to the chat window? And can I reach my agent there? That's all the questions are going to say. And if you come here, it says, "Oh, look at this. Open chat. Ctrl Alt AI." Let's see what happens if I hit Ctrl Alt I. There we go. That's a chat. Wait a minute. I see Claude code here because I had it before. If it was your first time, you would have to, you know, log in or everything, but I'm already logged in. Just so you know, all you got to do is find the chat window. And let's say here I could open a folder. But another thing I can do is I can go to Pinwist and then I get that agents MD. I'm going to rightclick. I'm going to copy his path. This is my agent MD file in Pinwist. Now watch this. Once I hit paste, I am now talking to my agent. Maybe you never used VS Code before. You don't need to know how to use it. You just need to know how to get to that chat window. Check Pinwist free version and Pinwist World. There it is y'all. Pinwist World. It doesn't matter where I go, the agent is the same. It doesn't matter if it's Claude, Codeex, or these local LLMs. You know how many local open- source stuff is going to come out? As more things come out, more agents that you can use, you no longer have to feel like you need to use Claude or you need to use Codeex. You can now experiment with other agents because all you're going to do is give them have them read the same file. No matter what you're using, no matter what company made the agent you're talking to, none of it matters anymore. Yeah, it matters if you want smarter ones, better ones, but with this file folder system, you are in control. I have full control over my agent that now just told me about Pinwist. I didn't mention anything. I just had it read the file and it knows what to say. What about other stuff? Cursor, right? Cursor came out. There's a thing called Composer 2.5. Before Claude really blew up. Composer was really up there with Claude. People were using Cursor with Composer. It's a respectable agent, right? And it just came out and said it's like 10 times cheaper than Chad GBT. And they got a free tier. So you want to use a free tier, you could open up Cursor. Now maybe you never use Cursor. It doesn't matter. What are you going to do? How do I get to a agent? Control shift L. This is my agent. It's composer. What am I going to do? Put that same agents MD file and hit enter. I've never used cursor before. I've never used composer 2.5 before, but I know how to open the app and I know how to look for a chat and I know how to paste my agents MD file. And what do you think is going to happen now? Doesn't matter what you're using the same thing. Now, this agent may be slower, maybe faster, it may be dumber, maybe smarter, but whatever work I do with this, that work will be saved to my files. It's not going to go back to Codeex's memory. It's not going to go back to Claude's memory. It's going to stay with me where it belongs. Now, yeah, they're getting a copy of it, but my organization is ran by me. Do not let AI run things for you. AI is not good at organizing. It's not good at this stuff, but you are. So, you need to be in charge. CWS upload. Pinwist World Concept. Pinwist World. There it is. I don't know if you can see it right here. Pinwist World Concept. See that? So, another thing that this can do for you is things like OpenClaw, Hermes Agent, right? All these companies are now coming out with desktop apps. And before you may have felt overwhelming, oh, Hermes, it sounds too complicated. But nowadays, it's not. If you follow the file and folder system, I can now install Hermes. I say, "Well, I never used Hermes before. How do you use this app?" I don't know. Let's open it and find out. You install it. You open it up. Now, I have Hermes agent. What are we going to do? We're going to go right back to that agent MD and hit enter. Now, let's see if Hermes, which is using ChadByt, by the way. Hermes, one cool thing about Hermes is you can load up a lot of different LLMs and it can use different ones for different occasions. That's what makes Hermes very special. Once you get into this stuff,

I won't be shocked if a lot of you actually use Hermes instead of Codeex. Not for everything, but when you want to do certain things like work around your organization and different things and you want to test out different LLMs, you want to test out different models, open source models, cheaper models, this is a very good way to do it. Hermes can be attached to a whole bunch of models at once. And look what it did right away. And by the way, another cool thing about Hermes, if you use Grock at all, Hermes is the only agent that can actually connect to your GROK subscription, not an API. So if you want to know one cool secret about how to do image generation almost unlimited and you can do a lot at one time, you can get a GROK subscription and use it with Hermes. Hermes, I believe, is the only place you can use it in this way. Gro has its own CLI, has its own thing, but I haven't used it. Some people say it's sort of okay, but Hermes can use Gro's image generation. So you could put your subscription, not an API, a subscription, and never have to pay per image. You can use Gro and make a whole bunch of images right here with Hermes agent. All right, check the free version. Upload the Chrome store Pinwist World VR like walkthrough. See this Pinwist World. And all I did was write it earlier, said hand off. And now anywhere I go, it doesn't matter where you are. You could be on your phone using Codeex remotely and your stuff was always up to date. Guys, I hope you guys are catching what's happening here. It's an agent that is never lost. An agent that knows everything there is to know. It knows where to grab the tools to do what it needs to do. And you can switch around and jump from app to app, from LLM to LLM. If you ever use CL codeex, you're like, is it getting dumber? This system will tell you that like, wait a minute, it never had a problem reading this before and now suddenly it can't follow directions. Time to switch your agent. So, this exposes when the companies make the agents dumber, which they clearly do. And that way, you know, well, I can switch over, right? And this way, you can't afford a \$200 uh plan on anything, but you might be able to afford three \$20 plans. And now you can jump back and forth this way, right? And now you can use anything. How many times have people seen this? You've seen this many times and you probably were intimidated. You probably said, "Well, I don't know how to use this, right? I'm going just go CD dot dot just goes up once." So now I'm in my C drive. So when you use something called a CLI, it's something that works here. I installed Codeex CLI. You might heard of anti-gravity CLI. You may have heard of Claude CLI. It just means it works in a terminal, right? So if I type the word Claude and I hit enter, Claude is going to start. All right? So, if I got Claude here, I can actually say something like, "Okay, let's work on some print on demand stuff." Now, I got a whole print on demand folder. Before, it would have easily found that folder. Now, I'm in my C drive. There's no agents file. Print on demand. Let's get into it. Before I start poking around, what do we do this session? See, that's asking questions. It has no clue what's going on. For context, I know we got automege pipeline. Nope, that's not what I want. I said I want to work on print on demand the stores. So I'm telling it I want to work on the stores. Got it. The stores. Let me find what we've got. So I'm working from the actual setup, not guessing. And now it's going to go and burn more tokens. It has no clue what I'm talking about, y'all. It has zero clue. So what I'm going to do, I'm going to hit escape to interrupt. I'm just going to copy his path. Agents MD and boom. Right. And there it goes. Which direction? And then it tells me everything I need to know about the stores. And now, you know, Pinterest is a separate thing. It's a Chrome extension. The actual store is a library with mockups and alpha designs. There some stuff that I'm working on where you can prompt the store in one prompt. Okay. So, now it's time for a pro tip. All right. You want to know, you know, how can you start making a system like this, right? Well, first of all, you want to go learn as much you can from this man right here. But here's a cool little tricks that can help you if you wanted to start doing this, right?

because you don't need to take someone else's system, right? You get the transcript from uh the Jake video and then you're going to talk to your AI. Maybe once this video is out, get a transcript from this video and then what you're going to do is you're going to help your AI know what it is that you want. Just take basically everything that he says in this video by just giving it the transcript. Right? I think I could use Whisper Flow for this. Hey, so I saw this video of Jake Van Clee and I think that his system that he uses is phenomenal and I want to build something like that basically where every, you know, folder is a project where the AI can be navigated through it using a router and map system. I want a system that uses these MD files and all the different folders that it gets routed to and so that it always knows where it's at or where it needs to go, what it needs to do, how it needs to do it, and what tools it needs and where those are. an AI that never gets lost using a routing system. Is this something you can help me build? Can we plan this out? I want to use this for my organization. Let's get into it. Right? So, you can plan, right? You don't have to start building it yet, but you could plan out the system and start saying, "Well, I want a system that does this. This is buildable without making over complicated. I'll treat the uploader script as the reference pattern." Then, yeah, this is exactly the kind of system you should build for your organization. You see this? What Jake is describing is not really in an app at first. it's a file system, blah blah. So the AI understands this. So if you want to build a system like this for yourself, just talk to your AI. You don't ever have to do any of the work yourself, but you just want your AI to understand it all. And then you as long as that AI writes this to its own agents MD, you will start off with an AI that then builds the rest of the system. So I'm not going to get into everything it does, but look, it's figuring it out. And this is what I did. You see this workspace system folder? This is the agents MD although it doesn't matter where I start. This is the one that knows everything. So the first thing I did for my organization is I built an agent whose sole purpose is to make the organization work this way. So before it even knew what I cared about, the first thing I said was we got to build this organizational system of files and folders and then we're going to use that to operate. And that way every time I did a new folder, so I don't make files and folders. What I do is I work right in here. So I do full audits and everything. So right here in this uh chat with this workspace system, this is where I might say, "Hey, I want to make a new project to learn how to do electrician work." This is my workspace agent, right? It already knows the system better than I know it because it helped me build it. And I built it using this AI. We spoke together. I said, "I want to route this way. Can we test the routing? Can we make sure it works?" This doesn't happen overnight and it doesn't happen, you know, easy. The best idea I can give you is start small. Don't try to build everything at once. You start small. Make it work for two folders. And once you get the okay, even though it's not going to get lost, it's only two folders. You still start there and then build up from it. And once your agent knows the rules, it'll start following those rules with everything it does and start testing it and all of that, making sure that the agent will never be lost. No matter what happens, it's ready to go. Now, anything extra it needs to know, I can go in that folder and then start talking to that agent about what I want to do there, right? But it onboarded it. It says, "Okay, we got to make sure." And now, if I, you know, read that agent's MD, what's it going to say? Root law. It's going to let it know. It has a map. It's saying this is electrician training law. Now, I can add more to this, but it gave it the basics that it needs to know. Do not present AI generated notes as permission. It just kind of made this up. electricity treat local electric code permits license. Look what it's doing. I didn't tell it to do any of this. It's making sure the AI has a certain kind of behavior, right? That's what I'm saying. I can go on and on, guys. There's so much I wanted to talk about in this video that it's just too much. Like, I'm not even halfway with what I wanted to show you. I'm not even

halfway and I'm realizing it's going to be too long. Crazy stuff, man. I can't tell you just how important all of this is to me. And the reason I'm making this video for you is because Jake's teach what he teaches is something that you cannot glance over. If you want to take power back in your life, if you want to make it so these companies cannot control you, they can't own you by saying, "Ah, you don't know what what what we're doing or how we're doing it." Yeah, we do. Start paying attention to all the news articles you see. What Codeex is doing, what Anthropic is doing. Because you know what they're doing? They're trying to build this system. This file and folder system is exactly what they're trying to build. They're trying they're spending millions and millions of dollars to build this. Oh, we need something that never forgets and just always has memory. Solved already, y'all. Solved. But it wasn't solved by them. So, don't need to be overwhelmed. You do this, you're ahead of the curve. And when that voice comes out, ooh, Jake says in a video, everyone's going to be talking in 6 months. And he was right. So, Open AI is making a voice agent that's just like Alexa. You seen Amazon Alexa? You could be sitting in your kitchen and just start talking to your agent. Think about this. When those voice agents come out, do you want an AI that they're spending millions of dollars trying to fix the memory that hopefully remembers something about you or your AI to live in this file and folder system? And you can sit in your kitchen and you could tell it anything you want. You can ask it anything you want and it never gets lost. If you build this system now, you're going to be thanking yourself when the voice agents really come out and we're no longer typing to anything. You go and you work on your business this way, but when you're out and about, you can now just use your voice. Your agent will never be lost. This is the future, ladies and gentlemen. This is where we're going. How Jake talks about. You're never This is going to still be valid in 10 years. So, this is the thing to learn. And it's not even AI. It's just a system that AI can follow perfectly. All these companies are trying to trick you. They don't want you to know this. This is why you got to support Jake Van Clee. He has his cliff notes. It's completely free if you want to go in there and find out what people are talking about. And another thing, they pay people to know the system. Companies pay thousands and thousands of dollars to get systems like this implement. So, there's so many reasons to learn this stuff. But y'all, this guy has taught us something so valuable. Please support him in anything he does because this is the kind of person I'm going to be following him. He's going to be the voice of reason. He's going to be my validation for anything that happens in the future. I'm going straight to Jake Van Clee to see what he says about it. Straight like that. Like what I'm showing you is about 10% of what Jake does. They do very intricate things with this. What he can do with this is a level that I can't wait to learn. And I hope you see this is why I made this video because this is what you should be doing. And while all these companies are all trying to show these shiny objects, you won't have that problem no more. They're trying to sell you the perfect agent all the time. Oh, the agent now has memory. The agent now does this. Agent now does that. But what if the best agent you ever use is the one you build yourself? Maybe. That's all I'm saying, y'all. If you got any value out of this, please like and subscribe, and I see you on the next one.

Watch, then make it concrete

After the introduction, write down one job you want your workspace to help with. Keep it small enough to review in one sitting.

A second way to see the method

This community breakdown from @iscalelabs introduces ideas you will meet throughout Foundation. Watch for the relationship between the task, the context you provide, and the result you check.

Then choose one small task of your own. You will use it in [Your First Folder](#).

Sepertinya kita punya push baru dari azis. coba clone duulu
https://github.com/totemocreativestudio/ai_affiliate.git